

UNIVERSITY OF CLAUDE · COURSE SYLLABUS

CSCE 640 — Quantum Algorithms

Interference is the resource, and the known advantages are few and specific

AT A GLANCE

Credits: 3 credit-equivalents · **Modules:** 13 · **Term:** Semester 12 (with CSCE 628 and CSCE 717)

Prerequisites: Linear algebra over the complex numbers, properly; CSCE 629 Analysis of Algorithms; CSCE 637 Complexity Theory for Module 12; CSCE 658 Randomized Algorithms, whose amplitude-amplification analogy runs throughout

Effort: 13–15 hours per week · 13 modules + 2 project weeks

Deliverable: A simulated implementation of one non-trivial quantum algorithm, with its classical baseline measured on the same instances, the advantage stated as a function of problem size, and the resource count that would be needed on hardware

Course description

This course is organised around a single honest claim: a quantum computer is a device that exploits interference among complex amplitudes, and the set of problems for which that is known to help is small, specific, and worth knowing exactly. Module 01 states the position and names the misconceptions, because the subject is surrounded by claims that do not survive contact with the mathematics.

The first third builds the machinery. States, measurement, gates, and circuits (Modules 02 and 03) — and then Module 04 isolates where the advantage actually comes from: arranging for the wrong answers' amplitudes to cancel, which is the one idea the whole subject rests on.

The second third is the algorithms that matter. The query model and Deutsch-Jozsa (Module 05) for the cleanest separation; Grover (Module 06) with its provably quadratic and no better limit; the quantum Fourier transform and phase estimation (Modules 07 and 08), which is where Shor's algorithm comes from and is the one genuinely dramatic result.

The third theme is the part the popular account omits. Noise, error correction, and the threshold theorem (Module 10) determine whether any of this runs, and the resource counts for useful instances are enormous — so Module 11 is about what the current generation of hardware can and cannot be expected to do, stated carefully.

And the closing position is about claims. Module 12 covers BQP and, more usefully, what is not believed to be in it, and Module 13 is about what a demonstrated advantage establishes — because 'a quantum computer solved this faster' is a claim about a specific problem, a specific classical baseline,

and a specific machine, and all three have to be named.

Learning outcomes

On finishing this course you should be able to do the following without reference material.

- 1 State what a quantum computer is, and refute the standard misconceptions.
- 2 Work with quantum states, superposition, and measurement.
- 3 Build and reason about quantum circuits.
- 4 Explain interference as the source of every known advantage.
- 5 Analyse algorithms in the query model.
- 6 Derive Grover's algorithm and its optimality.
- 7 Derive the quantum Fourier transform.
- 8 Explain phase estimation and Shor's algorithm.
- 9 Explain quantum simulation, the likeliest application.
- 10 Explain noise, error correction, and the threshold theorem.
- 11 Assess variational methods and near-term hardware claims.
- 12 Place BQP, and state what it is not believed to contain.
- 13 Claim a quantum advantage honestly.

Module schedule

One module per week for thirteen weeks, then two weeks for the final project. Each module is a slide deck, a set of lecture notes, mapped external lectures, and exercises that must run.

Wk	Module	Core question it answers
1	What a Quantum Computer Is	Does it try all the answers at once?
2	States and Measurement	Why can you not just read the amplitudes?
3	Gates and Circuits	What is the quantum equivalent of a NAND gate?
4	Interference	Where does the speedup actually come from?
5	The Query Model	One query, and you know the answer. What was the catch?
6	Grover's Algorithm	Why can't it be better than square root?
7	The Quantum Fourier Transform	How can a Fourier transform on 2^n points take n^2 gates?
8	Phase Estimation and Shor	Why does factoring reduce to finding a period?

Wk	Module	Core question it answers
9	Quantum Simulation	What is a quantum computer actually for?
10	Noise and Error Correction	You cannot copy a qubit. So how do you correct it?
11	Near-Term Hardware	What can you do with a hundred noisy qubits?
12	BQP and Its Neighbours	Can a quantum computer solve NP-complete problems?
13	Claiming a Quantum Advantage	A quantum computer did it faster. Faster than what?

Projects

Project 1 — Write the simulator, then run Grover (after Module 6)

Build the state vector machinery yourself, and measure what Grover actually buys.

Requirements

- **A state vector simulator from scratch** — state preparation, single-qubit and controlled gates, and measurement, in whatever language, with no quantum library.
- **Verified against hand calculations** on one- and two-qubit circuits, including an entangled state.
- **Deutsch-Jozsa implemented** (Module 05), with the oracle as a black box your algorithm may call but not inspect.
- **Grover implemented** (Module 06), with **the success probability plotted against the number of iterations** — including past the optimum, where it goes back down.
- **The iteration count compared to the predicted $\pi/4 \cdot \sqrt{N}$** , over several problem sizes.
- **And the classical baseline measured:** expected queries for random search, on the same instances.

Done means

- **The simulator written rather than imported** — which is the point of the project, because the amplitudes have to stop being abstract.
- **The success probability curve showing the decline** past the optimal iteration count — **which is the result that demonstrates you understand the rotation** rather than the recipe.
- **The quadratic speedup measured, not asserted**, with the constant factor reported.
- **And the entangled state verified by hand**, because **a simulator that gets product states right and entangled states wrong looks correct** until it does not.

Project 2 — One algorithm, honestly costed (after Module 12)

Implement something non-trivial, beat a real baseline, and count the resources.

Requirements

- **One algorithm from Modules 07 to 11** — phase estimation, a period-finding instance, a small simulation problem, or a variational method.
- **Derived by hand first**, with the derivation submitted — and **the implementation checked against it**.
- **A classical baseline implemented and measured** on the same instances — **the best classical method you can find, not a strawman**.
- **The scaling of both, measured over as many sizes as your simulator reaches**, with the fit stated.
- **A resource estimate for a useful instance: logical qubits, gate depth, and — using Module 10's overhead — physical qubits**.
- **And the honest claim** in Module 13 §2's form, including what your simulation could not reach.

Done means

- The baseline being the best classical method rather than a convenient one — which is what makes the comparison mean anything, and is where advantage claims most often fail (Module 13 §3).
- The resource estimate carried through to physical qubits, because logical-qubit counts understate the cost by three or four orders of magnitude (Module 10 §4).
- The scaling measured over a range rather than asserted from the asymptotic form.
- And the honest claim naming the problem, the baseline, and the machine — all three, which is the course's closing requirement and the thing most published advantage claims omit at least one of.

Assessment

Nobody grades this. The standard is therefore mechanical, and you apply it to yourself honestly or the credential is worth nothing.

Component	Weight	Standard
Module exercises	40%	Code runs and produces the specified output. Not 'I understand how I would do it.'
Projects	45%	In the portfolio repository, with a README showing results and a written note on what was hard.
Self-checks	15%	Answered in writing from memory. Recognition is not knowledge; writing exposes the difference.

Required materials

All free. Nothing in this course requires a purchase.

Resource	Role
Nielsen & Chuang — Quantum Computation and Quantum Information	The primary text, and the standard one. Modules 02 through 10 follow its development. Library copy; it is demanding and it is worth the demand.
Aaronson — Quantum Computing Since Democritus, and his lecture notes (free)	Modules 01, 12, and 13. The best available correction to the popular account, from somebody who works in the field and says plainly what is not known.
Kitaev, Shen & Vyalı — Classical and Quantum Computation	Modules 07, 08, and 12. The complexity-theoretic treatment, and the phase estimation development is especially clean. Library copy.
Preskill's lecture notes on quantum computation (free)	Modules 04, 10, and 11, free in full. The error correction and threshold material is the clearest written account, by the person who named the current era.

Resource	Role
The Qiskit textbook and documentation (free)	The implementation side , Modules 03 onward — and simulating a circuit you derived on paper is the fastest way to find out you derived it wrong.
Gidney & Ekerå — How to factor 2048 bit RSA integers (free)	Modules 10 and 13's resource counts . A careful, specific estimate of what Shor's algorithm would actually cost in physical qubits and hours, which is the number to quote.

Tooling

- **A simulator, and numpy is enough — a state of n qubits is a vector of 2 to the n complex amplitudes, and writing your own simulator in forty lines is the single most clarifying exercise in the course** (Module 02 §4).
- **Then Qiskit or Cirq**, for the circuit abstraction and the transpilation — **which makes the gate-count and connectivity constraints concrete** in a way a hand-rolled simulator hides.
- **And the honest ceiling: a laptop simulates roughly 25 to 30 qubits of general state, which is enough for every algorithm in this course and not enough for any useful instance of any of them** — a fact worth sitting with (Module 13 §1).
- **Real hardware is available free through cloud queues, and running a five-qubit circuit on it is worth doing once**, for Module 10's reason: **the output distribution will not be the one you derived**.
- **Pen and paper, unavoidably. Module 07's Fourier transform and Module 08's phase estimation have to be derived by hand** before any implementation makes sense.
- **And a classical baseline implementation for Project 2**, measured on the same instances — because **an advantage claim without a measured baseline is not a claim** (Module 13 §2).

Academic honesty, restated for a solo student

There is no one to cheat but yourself, which makes it easier, not harder. Two rules: copy code only after you have written a worse version yourself, and never let a working result stand in for an understood one. When you paste something you do not understand, write a TODO naming what you do not understand, and go back to it.